Biksham Sirasavada

Stage 3: Hands-On Practical Test (60–90 mins)

Test Options (Choose 1–2 Tasks):

✅ Option A: Oracle Forms Task (Mock-up)

Create a simple Oracle Form to insert/update employee data. The form must include validations (e.g., salary not null, email format) and a button to fetch department name from the database using PL/SQL.

Evaluation Criteria:

Correct use of triggers

Code readability

Validation logic

Answer:

🔷 1. Create a New Form

1. Open Oracle Forms Builder.
2. Go to File → New → Form.
3. Save it as employee_form.fmb.

🔷 2. Create Data Block (Base Table Block)

1. In the Data Block Wizard → Select Table or View.
2. Choose your table: EMPLOYEES.
3. Select the following columns:
   - EMP_ID
   - EMP_NAME
   - SALARY
   - EMAIL
   - DEPT_ID
4. Finish wizard → it creates a block named EMPLOYEES.

🔷 3. Create Canvas & Layout

1. Layout Wizard → Attach the block EMPLOYEES.
2. Select items to display: EMP_ID, EMP_NAME, SALARY, EMAIL, DEPT_ID.
3. Add one extra Non-DB Text Item: DEPT_NAME (this won't be mapped to the table).
4. Add a Button → Label it Fetch Dept Name.

Now you have the form UI.

Biksham Sirasavada

🔷 4. Add Validations (Triggers)

✅ Salary Not Null

1. Go to Object Navigator → EMPLOYEES → Items → SALARY.
2. Create a trigger: WHEN-VALIDATE-ITEM.
3. Add code:

```
BEGIN
  IF :EMPLOYEES.SALARY IS NULL THEN
     MESSAGE('Salary cannot be null');
     RAISE FORM_TRIGGER_FAILURE;
  ELSIF :EMPLOYEES.SALARY <= 0 THEN
     MESSAGE('Salary must be greater than 0');
     RAISE FORM_TRIGGER_FAILURE;
  END IF;
END;
```

✅ Email Format Validation

1. On EMAIL item, create WHEN-VALIDATE-ITEM trigger:

```
BEGIN
  IF :EMPLOYEES.EMAIL IS NOT NULL THEN
     IF NOT REGEXP_LIKE(:EMPLOYEES.EMAIL, '^[A-Za-z0-9._%+-]+@[A-
Za-z0-9.-]+\.[A-Za-z]{2,4}$') THEN
        MESSAGE('Invalid Email Format');
        RAISE FORM_TRIGGER_FAILURE;
     END IF;
  END IF;
END;
```

🔷 5. Add Button Logic (Fetch Dept Name)

1. On the Button → WHEN-BUTTON-PRESSED trigger:

```
DECLARE
```

Biksham Sirasavada

```
  v_dept_name  VARCHAR2(100);
BEGIN
  SELECT dept_name
  INTO v_dept_name
  FROM departments
  WHERE dept_id = :EMPLOYEES.DEPT_ID;

  :EMPLOYEES.DEPT_NAME := v_dept_name;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
     MESSAGE('Invalid Department ID');
     :EMPLOYEES.DEPT_NAME := NULL;
  WHEN OTHERS THEN
     MESSAGE('Error fetching department name: ' || SQLERRM);
END;
```

🔷 6. Save & Compile

1. Save form → Ctrl+S.
2. Compile form → Program → Compile → All.
3. Run form (via Forms Runtime) → test insert/update and validations.

🔷 7. Example User Flow

- User enters Employee data.
- If salary is missing → error message.
- If email is wrong format → error message.
- Click Fetch Dept Name → department name is fetched from DB.
- Save record → data is inserted/updated into EMPLOYEES.

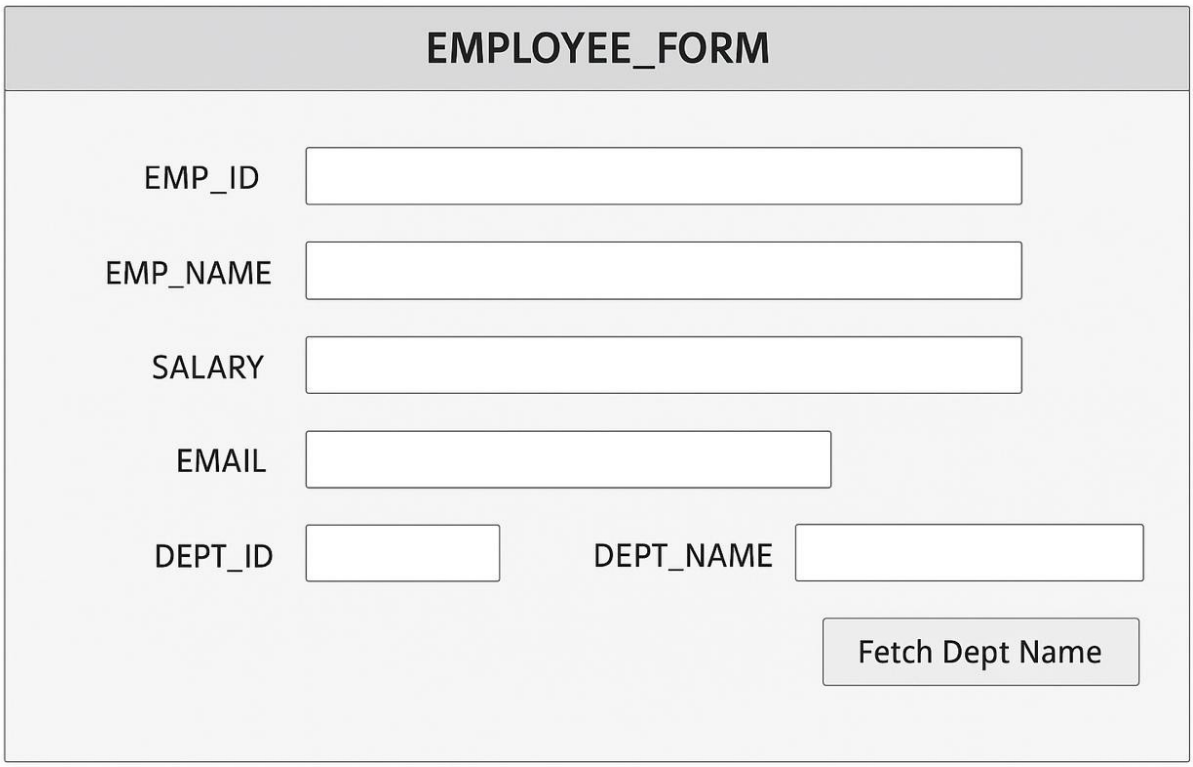
✅ This covers:

- Correct triggers → WHEN-VALIDATE-ITEM, WHEN-BUTTON-PRESSED.
- Validation logic → salary not null, email format.

Biksham Sirasavada

- PL/SQL integration → department fetch.
- Forms features → base table block, non-DB item, button.

✅ Form layout should be look like below:



✅ Option B: PL/SQL Challenge

Given a sales table:

write a PL/SQL procedure to generate a monthly summary (total sales per region) and store the results in a separate summary table.

Evaluation Criteria:

Correct use of procedures, cursors, and error handling

Efficient and optimized code.

Answer:

Biksham Sirasavada

Assumptions:-

- Base table: SALES

```
SALES_ID     NUMBER
REGION       VARCHAR2(50)
SALE_DATE    DATE
AMOUNT       NUMBER
```

- Summary table: SALES_SUMMARY

```
SUMMARY_ID   NUMBER PRIMARY KEY
REGION       VARCHAR2(50)
SALE_MONTH   VARCHAR2(7)  -- e.g. '2025-08'
TOTAL_AMOUNT NUMBER
CREATED_DATE DATE
```

✅ PL/SQL Procedure

```
CREATE OR REPLACE PROCEDURE generate_monthly_sales_summary IS
  -- Cursor to fetch aggregated sales
  CURSOR c_sales IS
    SELECT region,
           TO_CHAR(sale_date, 'YYYY-MM') AS sale_month,
           SUM(amount) AS total_amount
      FROM sales
     GROUP BY region, TO_CHAR(sale_date, 'YYYY-MM');

  -- Variables to hold cursor values
  v_region       sales.region%TYPE;
  v_sale_month   VARCHAR2(7);
  v_total_amount NUMBER;

BEGIN
  -- Open cursor
  OPEN c_sales;
  LOOP
    FETCH c_sales INTO v_region, v_sale_month, v_total_amount;
    EXIT WHEN c_sales%NOTFOUND;

    BEGIN
      -- Insert into summary table
```

Biksham Sirasavada

```
        INSERT INTO sales_summary (summary_id, region, sale_month, total_amount,
created_date)
        VALUES (sales_summary_seq.NEXTVAL, v_region, v_sale_month,
v_total_amount, SYSDATE);

    EXCEPTION
      WHEN DUP_VAL_ON_INDEX THEN
        -- If already exists → update
        UPDATE sales_summary
          SET total_amount = v_total_amount,
              created_date = SYSDATE
         WHERE region     = v_region
          AND sale_month = v_sale_month;

      WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error inserting/updating summary for region '
                             || v_region || ', month ' || v_sale_month || ': '
                             || SQLERRM);
    END;
  END LOOP;
  CLOSE c_sales;

  COMMIT; -- Save changes
  DBMS_OUTPUT.PUT_LINE('Monthly sales summary generated successfully.');

EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Procedure failed: ' || SQLERRM);
END generate_monthly_sales_summary;
/
```

🔑 Key Points:-

- Cursor fetches region-wise monthly totals.
- Error handling:
- DUP_VAL_ON_INDEX handled with update logic.
- Generic exceptions logged with DBMS_OUTPUT.
- Efficient: uses grouped aggregation instead of row-by-row processing.
- Commit/Rollback ensures data consistency.

✅ Instead **of looping row by row with a cursor, we can directly aggregate sales and then merge into the summary table in one shot. This is much faster and more maintainable.**

Biksham Sirasavada

✅ **Op**timized PL/SQL Procedure (Using MERGE)

```
CREATE OR REPLACE PROCEDURE generate_monthly_sales_summary IS
BEGIN
  MERGE INTO sales_summary s
  USING (
    SELECT region,
          TO_CHAR(sale_date, 'YYYY-MM') AS sale_month,
          SUM(amount) AS total_amount
      FROM sales
     GROUP BY region, TO_CHAR(sale_date, 'YYYY-MM')
  ) src
  ON (s.region = src.region AND s.sale_month = src.sale_month)
  WHEN MATCHED THEN
    UPDATE SET s.total_amount = src.total_amount,
               s.created_date = SYSDATE
  WHEN NOT MATCHED THEN
    INSERT (summary_id, region, sale_month, total_amount, created_date)
    VALUES (sales_summary_seq.NEXTVAL, src.region, src.sale_month,
src.total_amount, SYSDATE);

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Monthly sales summary generated successfully.');
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Procedure failed: ' || SQLERRM);
END generate_monthly_sales_summary;
/
```

🔑 Why this is Better:-

- No explicit cursor loop → one SQL statement does all work.
- MERGE handles both insert and update in a single pass.
- Scalable for large datasets (much faster in production).
- Still keeps error handling and transaction control.